

Explain code sorting

1. explain `sort(jobs.begin(),jobs.end(),cmp);` `bool cmp(job a,job b) { return a.profit>b.profit; }`2
2. is it in descending order?.....5



1. explain

```
sort(jobs.begin(),jobs.end(),cmp);
```

```
bool cmp(job a,job b)
{
    return a.profit>b.profit;
}
```

Let's break down what this C++ code does: 🖐️

◆ Code

cpp

```
sort(jobs.begin(), jobs.end(), cmp);

bool cmp(job a, job b)
{
    return a.profit > b.profit;
}
```

◆ Step-by-Step Explanation

1. **jobs** —

This is a container (usually a `vector<job>`) that holds objects of type `job`.

Example:

cpp

```
struct job {
    int id;
    int deadline;
    int profit;
};

vector<job> jobs = {
    {1, 2, 100},
    {2, 1, 50},
    {3, 2, 10}
};
```

2. `sort(jobs.begin(), jobs.end(), cmp);` —

This line sorts all elements in the `jobs` vector using the **custom comparison function** `cmp`.

- `jobs.begin()` — pointer/iterator to the first element.
- `jobs.end()` — pointer/iterator one past the last element.
- `cmp` — the comparison function that decides **how to order** the elements.

3. The comparison function `cmp`

```
cpp

bool cmp(job a, job b)
{
    return a.profit > b.profit;
}
```

This tells the sort algorithm:

"When comparing two jobs `a` and `b`, place `a` before `b` if `a.profit` is **greater** than `b.profit`."

✓ So the jobs will be **sorted in descending order of profit**.

◆ Example Result

Before sorting:

```
nginx

ID | Profit
1  | 100
2  | 50
3  | 10
```

After sorting with `cmp`:

```
nginx

ID | Profit
1  | 100
2  | 50
3  | 10
```

If you had a different order originally, this ensures the **highest profit comes first**.

◆ Summary

Line	Purpose
<code>bool cmp(job a, job b)</code>	Defines how two jobs are compared (by profit).
<code>sort(jobs.begin(), jobs.end(), cmp);</code>	Sorts all jobs in descending order of profit.

Would you like me to show how this fits into a **Job Sequencing Problem** (a common use case of this code)?



2. is it in descending order?

✓ **Yes**, it is in **descending order** of **profit**.

Here's why:

cpp

```
bool cmp(job a, job b)
{
    return a.profit > b.profit;
}
```

The `sort()` function in C++ works like this:

It places element **a** **before** element **b** if `cmp(a, b)` returns **true**.

So —

- If `a.profit > b.profit`, then **a** comes **before** **b**.
- Therefore, jobs with **higher profit** appear **earlier** in the list.

👉 **Result:**

The jobs are sorted in **descending order of profit** (highest → lowest).